Comp 512
Spring 2012

A Deeper Look at Data-Flow Analysis

+ vote on the lab

Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.

Students enrolled in Comp 512 at Rice University have explicit permission to make copies of these materials for their personal use.

Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.

Data-flow Analysis

Definition

Data-flow analysis is a collection of techniques for *compile-time* reasoning about the *run-time* flow of values

- We use the results of DFA to prove safety & identify opportunities
  - → Not an end unto itself
- Almost always involves building a graph
  - → Control-flow graph, call graph, or derivatives thereof
  - → Sparse evaluation graphs to model flow of values *(efficiency)*
- Usually formulated as a set of *simultaneous equations*
  - → Sets attached to nodes and edges
  - → Often use sets with a lattice or semilattice structure
- Desired result is usually *meet over all paths* solution
  - → "What is true on every path from the entry?"
  - → "Can this happen on any path from the entry?"

Data-flow Analysis

We have seen two data-flow problems: Dom and Live

Computing Dominators

- Domain is nodes in the flow graph being analyzed
- Simple set of data-flow equations

N is the set of nodes in the flow graph

Initialization

$$DOM(n_0) = \{ n_0 \}$$

$$DOM(n) = N, \forall n \neq n_0$$

Fixed-point equation

$$DOM(n) = \{ n \} \cup (\cap_{p \in preds(n)} DOM(p))$$

Can solve equations solve them with any data-flow solver

Data-flow Analysis

Computing Live variables

- Domain is the set of variable names in the procedure
- Data-flow equations are more complex

Initialization

$$LIVEOUT(n) = \emptyset, \ \forall n$$

Fixed-point equations

$$LIVEOUT(\mathrm{b}) = \bigcup_{s \in succ(b)} LIVEIN(\mathrm{s})$$

$$LIVEIN(b) = UEVAR(b) \cup (LIVEOUT(b) \cap \overline{VARKILL(b)})$$

where

UEVAR(b) is the set of names used in b before definition in b

VARKILL(b) is the set of names defined in b

Classic Algorithm: Round-robin Iterative Algorithm

$DOM(n_0) \leftarrow \{ n_0 \}$

for i $\leftarrow$ 1 to $|N|$

  $DOM(n_i) \leftarrow \{ N \}$

change $\leftarrow$ true

while (change)

  change $\leftarrow$ false

  for i $\leftarrow$ 0 to $|N|$

    $TEMP \leftarrow \{ n_i \} \cup (\cap_{p \in pred(ni)} DOM(p))$

if $DOM(n_i) \neq TEMP$ then

    change $\leftarrow$ true

    $DOM(n_i) \leftarrow TEMP$

Just the fixed-point equation

Very Simple Algorithm

- Halts when *DOM* sets stop changing
- Makes successive sweeps over the nodes in some fixed order

  $\rightarrow$ i $\leftarrow$ 0 to $|N|$

Solving a Data-flow Problem

To compute Dominator sets

- We need to build the control-flow graph
  - → Defines *predecessors* and *successors*
- Run the round-robin worklist algorithm
  - → Initializes *DOM(n)* for each node *n*
  - → Iterates until it reaches a fixed point (*e.g.*, *DOM* stabilizes)

To solve another data-flow problem

- Replace the initialization step and the fixed-point equation
- Fixed-point equation includes direction of propagation
  - → *Predecessors* or *successors*, as needed

To explain data-flow analysis, Kildall introduced a lattice-theoretic model. Kam & Ullman (among others) developed specific formulations for *iterative* data-flow algorithms

See J.B. Kam and J.S. Ullman, "Global Data Flow Analysis and Iterative Algorithms", *JACM* 23(1), January 1976, pp. 158-171

Classic Algorithm: Round-robin Iterative Algorithm

$DOM(n_0) \leftarrow \emptyset$

for i $\leftarrow$ 1 to $|N|$

  $DOM(n_i) \leftarrow \{ N \}$

change $\leftarrow$ true

while (change)

  change $\leftarrow$ false

  for i $\leftarrow$ 0 to $|N|$

    $TEMP \leftarrow \{ n_i \} \cup (\cap_{p \in pred(ni)} DOM(p))$

if $DOM(n_i) \neq TEMP$ then

    change $\leftarrow$ true

    $DOM(n_i) \leftarrow TEMP$

Just the fixed-point equation

Questions we must ask

- Termination: does it halt?
- Correctness: what answer does it produce?
- Speed: how quickly does it find that answer?

Data-flow Analysis

The basics

- Data-flow sets are drawn from a semi-lattice, L, of facts
- Sets are modified by transfer functions, f_i, that model effect of code on contents of the sets
  - → Function space of all possible transfer functions is F
- Properties of L and F govern termination, correctness, & speed

To reason about the properties of a (*proposed*) data-flow problem, we cast it into a lattice-theory framework and prove some simple theorems about the problem.

Data-flow Analysis

Limitations

1. Precision – *"up to symbolic execution"*
   → Assume all paths are taken
2. Solution – cannot afford to compute MOP solution
   → Large class of problems where MOP = MFP= LFP
   → Not all problems of interest are in this class
3. Arrays – treated naively in classical analysis
   → Represent whole array with a single fact
4. Pointers – difficult (*and expensive*) to analyze
   → Imprecision rapidly adds up
   → Need to ask the right questions

Good news:

Simple problems can carry us pretty far

Summary

For scalar values, we can quickly solve simple problems

*

Data-flow Analysis

Semilattice

A semilattice $L = (S, \wedge)$ is a set S and a meet operation $\wedge$ such that,

$\forall\ a, b, \&\ c \in L$:

1. $a \wedge a = a$
2. $a \wedge b = b \wedge a$
3. $a \wedge (b \wedge c) = (a \wedge b) \wedge c$

The meet operator combines the sets when two paths converge, or meet.

$\wedge$ imposes a partial order on L, $\forall\ a, b, \&\ c \in L$:

1. $a \geq b \Leftrightarrow a \wedge b = b$
2. $a > b \Leftrightarrow a \geq b$ and $a \neq b$

A semilattice has a bottom element, denoted $\perp$

1. $\forall\ a \in L, \perp \wedge a = \perp$
2. $\forall\ a \in L$, a $\geq \perp$

Sometimes we work with a lattice, which has a top element, denoted

$\forall\ a \in L, \top \wedge a = a$

and another operator, called join $\vee$

Data-flow Analysis

How does this relate to data-flow analysis?

- Choose a semilattice to represent the facts
- Attach a meaning to each $a \in L$

  Each $a \in L$ is a distinct set of known facts
- With each node n, associate a function $f_n : L \rightarrow L$

  f_n models behavior of code in block corresponding to n
- Let F be the set of all functions that the code might generate

2^N is the powerset of N, the set of all subsets of N

Example — DOM

- Semilattice is $(2^N, \wedge)$, where N is the set of nodes in the flow graph and $\wedge$ is $\cap$, and $\perp$ is $\emptyset$
- For a node n, f_n has the form $f_n(x) = x$

World's simplest data-flow equation

Data-flow Analysis

How does this relate to data-flow analysis?

- Choose a semilattice to represent the facts
- Attach a meaning to each $a \in L$

  Each $a \in L$ is a distinct set of known facts
- With each node n, associate a function $f_n : L \rightarrow L$

  f_n models behavior of code in block corresponding to n
- Let F be the set of all functions that the code might generate

Example — Live

- Semilattice is $(2^{Vars}, \wedge)$, where *Vars* is the set of names in the code and $\wedge$ is $\cup$, and $\perp$ is *Vars*
- For a node n, f_n has the form $f_n(x) = a \cup (x \cap b)$, where a & b are constants (*UEVAR & VARKILL respectively*)

A particularly common form for a data-flow equation

Iterative Data-flow Analysis

Termination

- If every $f_n \in F$ is monotone, *i.e.*, , $f(x \wedge y) \leq f(x) \wedge f(y)$, <u>and</u>
- If the lattice is <u>bounded</u>, *i.e.*, every descending chain is finite
  - Chain is sequence $x_1, x_2, \ldots, x_n$ where $x_i \in L$, $1 \leq i \leq n$
  - $x_i > x_{i+1}$, $1 \leq i < n \Rightarrow$ chain is <u>descending</u>

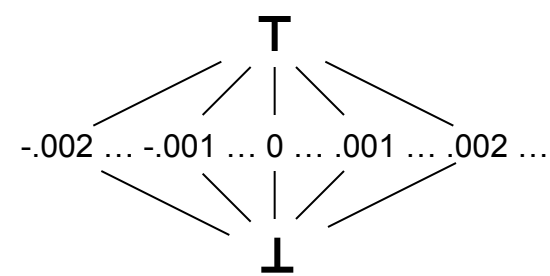
Finite lattice, bounded descending chains, & monotone functions $\Rightarrow$ termination

Both DOM & LIVE have monotone transfer functions & finite (bounded) semilattices.

Then

- The set at each node can only change a finite number of times
- The iterative algorithm must halt on an instance of the problem ♦

- Any finite semilattice is bounded
- Some infinite semilattices are bounded



Real constants

Iterative Data-flow Analysis

Correctness

(What does it compute?)

- If every $f_n \in F$ is monotone, *i.e.*, $f(x \wedge y) \leq f(x) \wedge f(y)$, <u>and</u>
- If the semilattice is <u>bounded</u>, *i.e.*, every descending chain is finite
  - Chain is sequence $x_1, x_2, \ldots, x_n$ where $x_i \in L$, $1 \leq i \leq n$
  - $x_i > x_{i+1}$, $1 \leq i < n \Rightarrow$ chain is <u>descending</u>

$f^k(x)$ is the application of f to x k times

Given a bounded semilattice S and a monotone function space F

- $\exists\, k$ such that $f^k(\bot) = f^j(\bot)\ \forall\, j > k$
- $f^k(\bot)$ is called the least fixed-point of f over S
- If L has a $\top$, then $\exists\, k$ such that $f^k(\top) = f^j(\top)\ \forall\, j > k$ and $f^k(\top)$ is called the maximal fixed-point (MFP) of f over S

optimism

MFP is largest fixed point, defined by ≥; LFP is smallest fixed point.

Iterative Data-flow Analysis

Correctness

- If every $f_n \in F$ is monotone, *i.e.*, $f(x \wedge y) \leq f(x) \wedge f(y)$, <u>and</u>
- If the lattice is <u>bounded</u>, *i.e.*, every descending chain is finite
  - → Chain is sequence $x_1, x_2, \ldots, x_n$ where $x_i \in L$, $1 \leq i \leq n$
  - → $x_i > x_{i+1}$, $1 \leq i < n$ ⇒ chain is <u>descending</u>

Then

- The round-robin algorithm computes a least fixed-point (LFP)
- The uniqueness of the solution depends on other properties of *F*

- Unique solution ⇒ it finds the one we want
- Multiple solutions ⇒ we need to know which one it finds

Iterative Data-flow Analysis

Correctness

- Does the iterative algorithm compute the desired answer?

Admissible Function Spaces

1. $\forall f \in F, \forall x,y \in L, f(x \wedge y) = f(x) \wedge f(y)$
2. $\exists f_i \in F$ such that $\forall x \in L, f_i(x) = x$
3. $f,g \in F \; \exists h \in F$ such that $h(x) = f(g(x))$
4. $\forall x \in L, \exists$ a finite subset $H \subseteq F$ such that $x = \wedge_{f \in H} f(\perp)$

If meet does not distribute over function application, then the fixed point solution *may not be* unique. The iterative algorithm will find a LFP.

If F meets these four conditions, then an instance of the problem will have a unique fixed point solution *(instance $\Rightarrow$ graph + initial values)*

$\Rightarrow$ LFP = MFP = MOP

$\Rightarrow$ order of evaluation does not matter

Both DOM & LIVE meet all four criteria

*

Iterative Data-flow Analysis

If a data-flow framework meets those admissibility conditions then it has a unique fixed-point solution

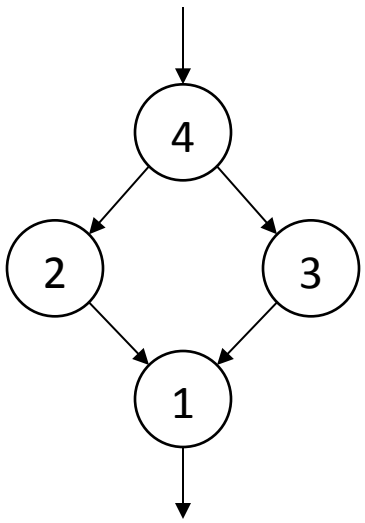
- The iterative algorithm finds the (best) answer
- The solution does not depend on order of computation
- Algorithm can choose an order that converges quickly

Intuition

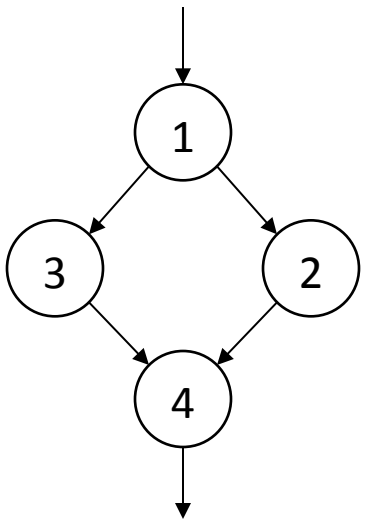
- Choose an order that propagates changes as far as possible on each "sweep"
  - → Process a node's predecessors before the node
- Cycles pose problems, of course
  - → Ignore back edges when computing the order?

*

Ordering the Nodes to Maximize Propagation



Postorder



Reverse Postorder

N+1 - *postorder number*

- Reverse postorder visits predecessors before visiting a node
- For backward problems, compute reverse postorder on the reverse CFG
  - → Reverse preorder is not necessarily the same

See Exercise 9.4 in EaC2e

Data-flow Analysis

Complexity

- For a problem with an admissible function space & a bounded semilattice,
- If the functions all meet the *rapid condition*, *i.e.*,

$$\forall f,g \in F, \forall x \in L, f(g(\perp)) \geq g(\perp) \wedge f(x) \wedge x$$

Sets stabilize in two passes around a cycle.

then, a round-robin, reverse-postorder iterative algorithm will halt in d(G)+3 passes over a graph G

Each "pass" does **O**(*E*) meets and **O**(*N*) other operations

d(G) is the loop-connectedness of the graph w.r.t a *DFST*

→ Maximal number of back edges in an acyclic path

→ Several studies suggest that, in practice, d(G) is small (*<3*)

→ For most CFGs, d(G) is independent of the specific *DFST*

Kam and Ullman, JACM, 1976. The paper gives a couple of equivalent formulations of the *rapid condition.*

Data-flow analysis

What does this mean?

- Reverse postorder
  - → Number the nodes in a postorder traversal
  - → Reverse the order
- Round-robin iterative algorithm
  - → Visit all the nodes in a consistent order (RPO)
  - → Do it again until the sets stop changing

So, these conditions are easily met

- → Admissible framework, rapid function space
- → Round-robin, reverse-postorder, iterative algorithm

⇒The analysis runs in (*effectively*) linear time

Data-flow Analysis

How do we use these results?

- Prove that data-flow framework is admissible & rapid
  - → Its just algebra
  - → Most (but not all) global data-flow problems are rapid
  - → This is a property of *F*

- Code up the iterative algorithm
  - → World's simplest data-flow algorithm
  - → Other versions (worklist) have similar behavior

This lets us ignore most of the other data-flow algorithms in 512

DOM, LIVE, and most global data-flow problems are *admissible* & *rapid*

Comp 512 Lab

Time to Make Decisions

Options:

- Each team does the same basic study with a different compiler
  - → Standardize on language(s), benchmarks, and tests
  - → Vary the compilers
- Each team does a similar, but different, study
  - → Standardize on methodology
  - → Teams choose a language, compiler, and benchmarks

Either option should lead to an interesting learning experience

Data-flow Analysis

How do we use these results?

- Prove that data-flow framework is admissible & rapid
  - → Its just algebra
  - → Most (but not all) global data-flow problems are rapid
  - → This is a property of *F*

- Code up the iterative algorithm
  - → World's simplest data-flow algorithm
  - → Other versions (worklist) have similar behavior

This lets us ignore most of the other data-flow algorithms in 512

How about some examples of problems that are *not* rapid?

Flow-Insensitive Interprocedural Summary Problems

MOD: $v \in$ MOD(s) $\Leftrightarrow$ executing s might change v's value

REF: $v \in$ REF(s) $\Leftrightarrow$ executing s might change v's value

These formulations can be solved with a flow-insensitive method

The equations

$$\text{GMOD}(p) = \text{IMOD}(p) \cup \cup_{e = \langle p,q \rangle} b_e(\text{GMOD}(q))$$

$$\text{DMOD}(e) = b_e(\text{GMOD}(q))$$

Where

GMOD(p): variables that may be modified by an execution of p

IMOD(p): variables that may be modified locally in p

DMOD(s): variables that may be modified by call site s, ignoring aliasing

b_e: a function that inverts the formal parameter $\rightarrow$ actual parameter binding

These functions do not have the "classic" form:

$$a = b \cup (c \cap d)$$

b_e makes the problem *non-rapid* on the call graph.

Solve on either the call graph or the binding graph β

Cooper & Kennedy, PLDI 88, June, 1988

Flow-Insensitive Interprocedural Summary Problems

MOD: $v \in$ MOD(s) $\Leftrightarrow$ executing s might change v's value

REF: $v \in$ REF(s) $\Leftrightarrow$ executing s might change v's value

Worst case examples

```
fee : procedure (a, b, c, d, e);
        integer a, b, c, d, e, f;
        …
        f = 0;
        ….
        call fee(b, c, d, e, f);
        end;
```

The "shift register"

Flow of values through the procedure

Interprocedural flow from binding at the call site

Flow-Insensitive Interprocedural Summary Problems

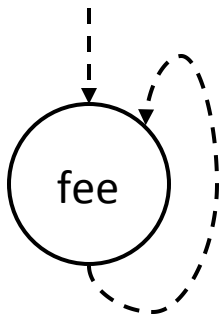
MOD: $v \in$ MOD(s) $\Leftrightarrow$ executing s might change v's value

REF: $v \in$ REF(s) $\Leftrightarrow$ executing s might change v's value

Worst case examples

```
fee : procedure (a, b, c, d, e);
        integer a, b, c, d, e, f;
        …
        f = 0;
        ….
        call fee(b, c, d, e, f);
        end;
```

The "shift register"



Call graph (fragment)

How many iterations does it take to determine that the value of a can change due to a call to fee()?

The answer depends on the number of edges in the cycle of bindings — which is unrelated to the call graph.

(Add another 5 parameters)

Flow-Insensitive Interprocedural Summary Problems

To solve these problems quickly:

1. Build a graph, β, that reflects the binding patterns of the parameters
   a. Node for each formal parameter and each actual parameter
   b. Edges for each binding at each call site
2. MOD & REF for parameters are now reachability on β
3. Solve the problem for global variables with a simple data flow equations

$$\text{GMOD}_G(p) = \text{IMOD}_G(p) \cup_{e = (p,q) \in CG} \text{GMOD}_G(q)$$

$$\text{DMOD}_G(e=(p,q)) = \text{GMOD}_G(q)$$

Steps 1 & 2 are linear in the size* of β.

Step 3 takes d(CG) + 3 passes, in an RPO traversal of the RCFG

Arguably, linear time or near linear time

* β is linearly larger than the call graph, as long as MaxParms is bounded.

Example

|CONSTANTS| = |*variables*|

Transformation: Global Constant Folding

- Along every path to *p*, *v* has same known value
- Specialize computation at *p* based on *v*'s value

Data-flow problem: Constant Propagation

Domain is the set of pairs $<v_i,c_i>$ where v_i is a variable and $c_i \in C$

$$\text{CONSTANTS}(b) = \wedge_{p \in preds(b)} f_p(\text{CONSTANTS}(p))$$

- $\wedge$ performs a pairwise meet on two sets of pairs
- $f_p(x)$ is a block specific function that models the effects of block p on the $<v_i,c_i>$ pairs in *x*

Form of *f* is quite different than in *LIVE*

Constant propagation is a forward flow problem

Classic formulation before Reif-Lewis

Example

Meet operation requires more explanation

- $c_1 \wedge c_2 = c_1$ if $c_1 = c_2$, else $\perp$ (*bottom & top as expected*)

What about f_p ?

- If p has one statement then

  $x \leftarrow y$ with CONSTANTS(p) = {…<x,i_1>,…<y,i_2>…}

  then f_p(CONSTANTS(p)) = CONSTANTS(p) - <x,i_1> + <x,i_2>

  $x \leftarrow y$ *op* z with CONSTANTS(p) = {…<x,i_1>,…<y,i_2>,…<z,i_3>…}

  then f_p(CONSTANTS(p)) = CONSTANTS(p) - <x,i_1> + <x,i_2 *op* i_3>

- If p has n statements then

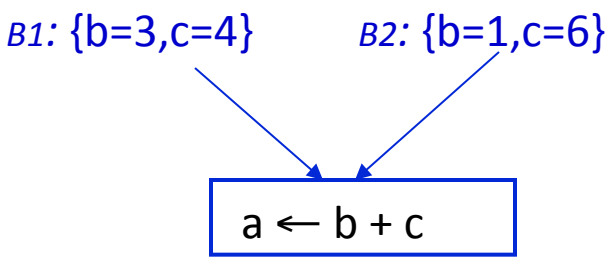
  $f_p(\text{CONSTANTS}(p)) = f_n(f_{n-1}(f_{n-2}(\ldots f_2(f_1(\text{CONSTANTS}(p)))\ldots)))$

  where f_i is the function generated by the ith statement in p

f_p interprets p over CONSTANTS

Some problems are not admissible

Global constant propagation



- Function "f" models block's effects
- f(*B1*) = {a=7,b=3,c=4}
- f(*B2*) = {a=7,b=1,c=6}
- f(*B1*∧*B2*) = Ø

- First condition in admissibility

  $\forall f \in F, \forall x,y \in L, f(x \wedge y) = f(x) \wedge f(y)$
- Constant propagation is not admissible
  - → Kam & Ullman time bound does not hold
  - → There are tight time bounds, however, based on lattice height
  - → Require a variable-by-variable formulation …
  - → The problem has multiple fixed-point solutions!

Comp 512 Lab

Some Suggested Benchmarks

Name	*Language*	*Codes*	*SLOCs*	*Source*
NoFib	Haskell	~30	8 to 1,500	Glasgow
OOPack	C++	4	Kernels	LBL
CSIBE	C	19	?	U Szeged
MiBench	C	38		U Mich
Polyhedron	Fortran 90	17	?	Polyhedron Labs
MediaBench I	C	11		Consortium
DaCapo	Java	14		DaCapo Project
Caffeine	Java	9		Pendragon Software
Chrome	C/C++	1	Lots	Google
Individual Codes	C?	Avida (Mich ST) Owl (RICE)		

Will Partain

Comp 512 Lab

Some Suggested Compilers

- gcc
- Llvm
- Open64
- IBM XL C compiler for POWER7
- Safe C compiler for web applications (google project?)
- GHC — the Glasgow Haskell Compiler